

# Conv1

## Assumptions

**Input:** 8 8 1

- greyscale image
- each pixel will be 8-bit int

**Kernel:** 3 \* 3

- Stride -> 2
- Padding -> 1 ( to make sure i get all the border values and don't lose data )
- 1 channel -> greyscale image
- 9 bytes of weights -> will be stored in the cache(local memory) -> Weight Stationary Dataflow

**Constraints:**

- 4 MACs -> each 8-bit \* 8-bit -> 16-bit accumulate & output -> overflow policy - round/clip (assumption)
- 16 bytes local memory -> SRAM (cache memory assumption)
- 1 KB/s external memory transfer bandwidth -> 1024 B/s
- Inputs and outputs will be streamed

## Calculations

### Cycles and patch

Number of MAC operations ->  $16 \cdot 9 = 144$

*number of MAC -> 4*

*Cycles ->  $144/4 = 36$  cycles + 32 cycles -> 68 cycles*

*for each output 9 MAC operations,*

*pairwise reduction with 2 MACs will be used to complete the partial sum calculations and complete the operation -> this will take 2 cycles*

$2 \cdot 16 = 32$  cycles

While reading later i found that i had missed the final accumulate stage in these calculations therefore MAC cycle frequency is  $68 \cdot 12.8 = 870.4 = 871$  Hz (approx)

Cycle 0:  $MAC0 \leftarrow a0w0$   $MAC1 \leftarrow a1w1$   $MAC2 \leftarrow a2w2$   $MAC3 \leftarrow a3w3$  (compute 0..3)

Cycle 1:  $MAC0 \leftarrow a4w4$   $MAC1 \leftarrow a5w5$   $MAC2 \leftarrow a6w6$   $MAC3 \leftarrow a7w7$  (compute 4..7)

Cycle 2:  $MAC0 \leftarrow a8w8$   $MAC1$  idle  $MAC2$  idle  $MAC3$  idle (compute 8)

Cycle 3:  $MAC0 \leftarrow ACC0+ACC1$   $MAC1 \leftarrow ACC2+ACC3$   $MAC2/3$  idle (pairwise adds)

Cycle 4:  $MAC0 \leftarrow MAC0 + MAC1$  (final add  $\rightarrow$  final\_sum)

therefore  $5 \times 16 = 80$  cycles

Input =  $8 \times 8 \times 1$       kernel =  $3 \times 3$       stride = 2  
Output =  $4 \times 4$       Padding = 1

$\rightarrow 16$  # of patches.  
for each output = 9 MACs  
Total operations =  $9 \times 16 = 144$ .

4 MACs, 16 byte SRAM, 1 kB/s external

For each input = 8 bits int.  
only 1 channel.

16-bit O/P.

using weight-stationary  $\rightarrow 9$  B  
 $\therefore 7$  B left of 16.

$$O/P = \frac{(8 - 3 + (2 \times 1))}{2 + 1} = 4$$
  
 $\therefore 4 \times 4 = 16$  pixels

Cycles needed  
assuming each MAC takes 1 cycle & there are 4  
 $144 / 4 = 36$  cycles per image.

## Memory and transfer speeds

input  $\rightarrow 8 \times 8 \times 1 = 64$  bytes  $\rightarrow$  read

output  $\rightarrow 4 \times 4 = 16$  bytes  $\rightarrow$  write

total is = 80 bytes

weights  $\rightarrow 3 \times 3 = 9$  bytes  $\rightarrow$  read once and stored in SRAM

bandwidth  $\rightarrow 1024 / 80 = 12.8$  images/s

Memory

I/P = 64 B.  $\rightarrow$  read + 1.9 B read.  
O/P = 16 B  $\rightarrow$  write

bandwidth

~~1024~~ 1024 180 = 12.8 images

$\therefore$

MAC runs at  $36 \times 12.8 = 460.8 \approx 461 \text{ kHz}$

## Padding and stride

There will be 16 patches to ensure all data points are covered



$8 \times 8 \times 1$  → grayscale.

weights → 8 bits  
kernel →  $3 \times 3 \rightarrow 9$

stride = 1

$P = 0$

2 overlap

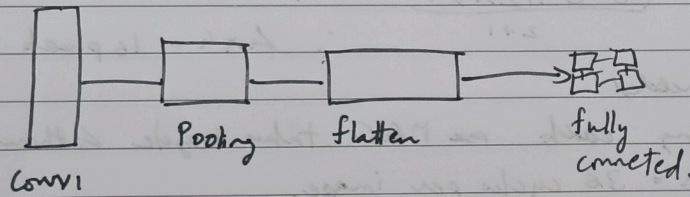
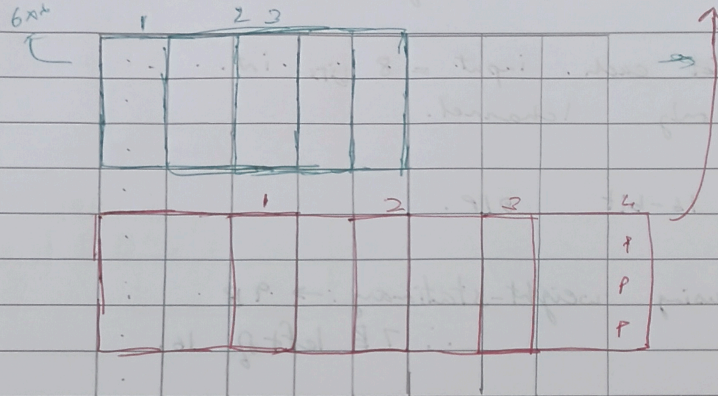
↓  
 $6 \times 6$

Stride = 2

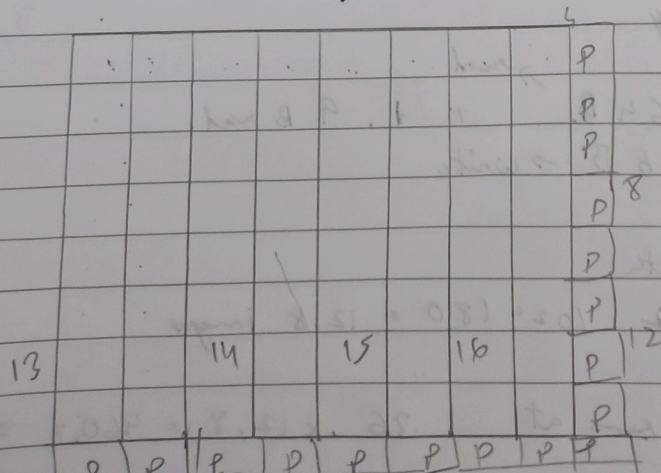
$P = 1$ ?,  $P = 0$

1 overlap.

↓  
 $4 \times 4$



cells that need padding → 4, 8, 12, 13, 14, 15, 16.



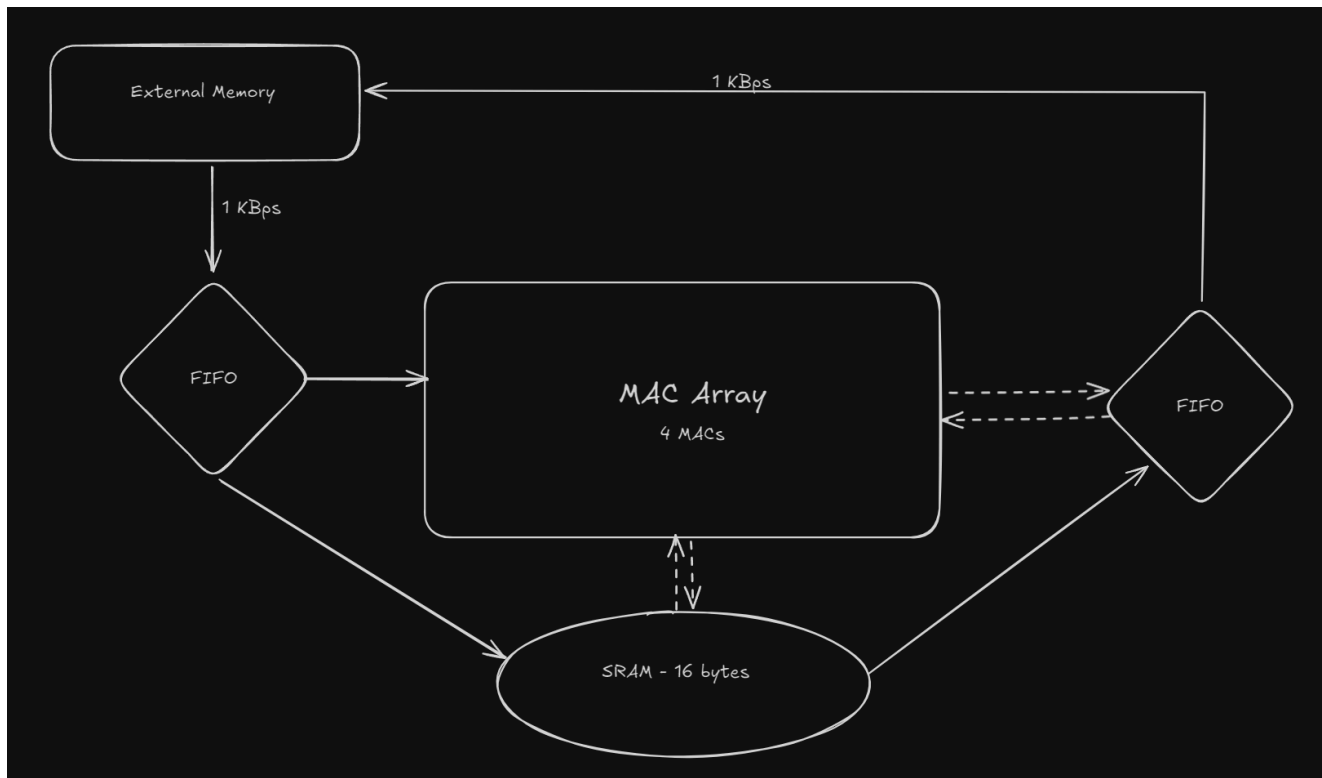
## Architecture

### System Overview:

The below figure shows the Conv1 system, where the main constraint is the tiny 16-byte on-chip SRAM. This memory acts only as a scratchpad and permanently stores the  $3 \times 3$  (9-byte)



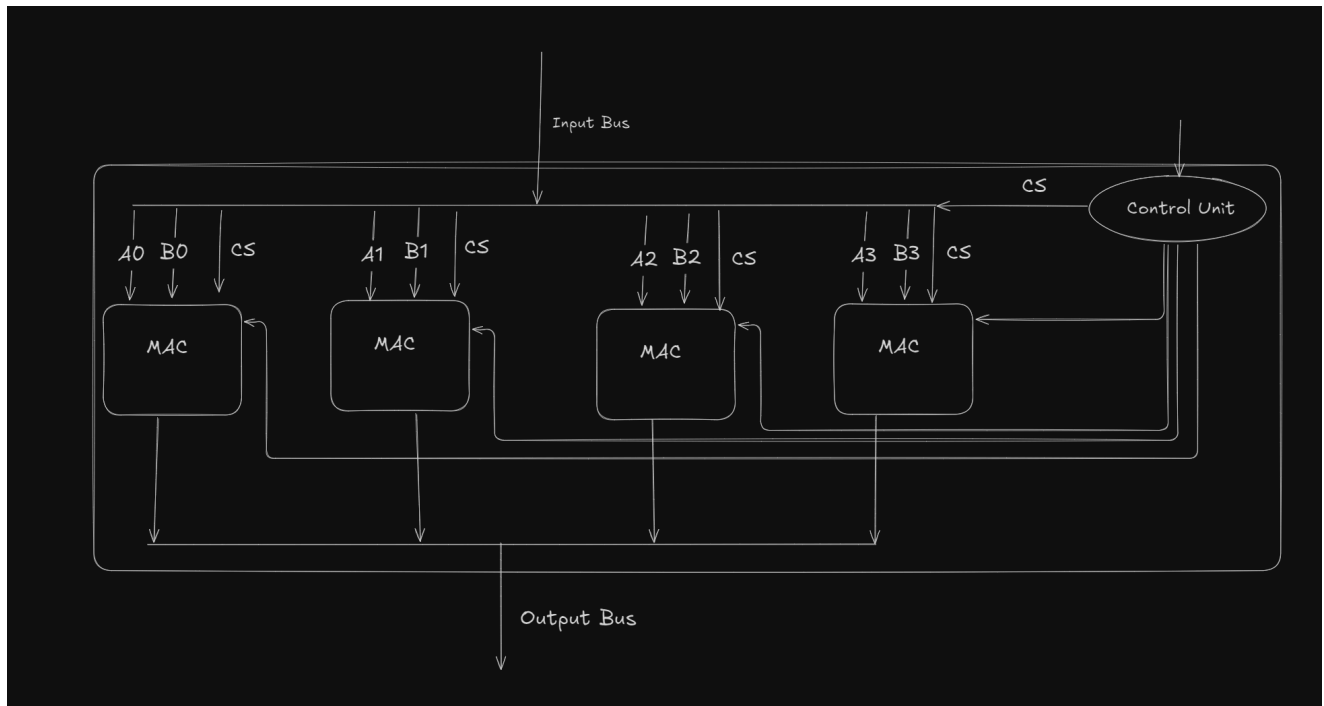
filter weights, leaving almost no space for activations or patch buffering. As a result, all input pixels must stream directly through the MAC array, making the design weight-stationary and fully streaming. The input and output FIFOs form the real synchronization layer, smoothing the flow between external memory and the compute core. The diagram reflects this lightweight loop: external memory → FIFOs → MAC array → FIFOs → external memory.



## MAC Array structure:

The below figure shows the organization of the four-lane MAC array and its associated control plane. The array operates in a SIMD-style configuration, where a single Control Unit broadcasts the relevant timing and operation signals to all MAC units simultaneously. An input bus delivers pixel values to all lanes in parallel, enabling concurrent multiply-accumulate operations across the four MACs. The dedicated CS (control-signal) lines shown in the figure carry cycle-accurate commands such as register write-enable, mux-select, and accumulation triggers. These signals allow the Control Unit to coordinate the serial-reduction phase, where specific MACs are selectively activated to write computed partial sums back to

the shared SRAM in a controlled sequence.



## MAC Internal View

The below figure illustrates the internal micro-architecture of a single MAC unit. Each MAC contains input registers feeding a combinational multiplier and adder, with the results captured in a dedicated accumulative register. A 2:1 multiplexer in the feedback loop allows the Control Unit to toggle between two operating modes: accumulation mode, where partial sums continue building over several cycles, and reset mode, where the accumulator is cleared without incurring additional compute overhead. This structure supports an output-stationary flow, meaning that partial sums remain local within the MAC until the reduction phase completes. The arrangement minimizes data movement and avoids unnecessary



SRAM traffic while keeping the reduction logic straightforward.

